

Build, share, and operate agents with control

A sequenced launch plan for Microsoft 365 Copilot and Copilot Studio agents: establish ownership and data boundaries first, then govern delivery, publishing, evidence, and consumption.

Start before build

Personal, read-only productivity agents can use a low-risk lane. Agents shared across a department, connected to business actions, or treated as mission-critical should use an IT-managed Copilot Studio lifecycle.

PHASE 1 | FOUNDATION
Establish control

- 1

Classify and assign owners

Classify personal, departmental, or mission-critical use. Record the business owner, technical owner, backup owner, support contact, audience, risk, cost center, and next review date. [Governance zones >](#)
- 2

Create lifecycle environments

Use separate development, test, and production environments for shared agents. Attach Microsoft Entra security groups, assign least-privilege roles through group teams, and keep production authoring access small. [Environment strategy >](#)
- 3

Enforce DLP and identity

Require Microsoft Entra authentication. Allow only approved knowledge types, connectors and tools, HTTP endpoints, skills, and channels. Test that policy violations block publishing before makers proceed. [Configure data policies >](#)

PHASE 2 | DELIVERY
Release safely

- 4

Review data and connections

Review and correct SharePoint permissions before connecting a site as an agent knowledge source. Document each source, endpoint, connection identity, secret, third-party processor, and data-residency exception. Prefer user-context access and least-privilege identities. [SharePoint knowledge access >](#)
- 5

Standardize build and test

Require scoped instructions, approved tools, safe failure behavior, expected outcomes, abuse cases, a human or support path, and test evidence. Validate with limited users before production review. [Test your agent >](#)
- 6

Promote through controlled ALM

Package agents and dependencies in custom solutions. Use connection references and environment variables; promote the same version through dev, test, and prod with GCC-supported PAC CLI or Azure DevOps Build Tools. [Solutions and ALM >](#)

PHASE 3 | OPERATIONS
Publish and sustain

- 7

Gate sharing and publishing

Separate Editor from Viewer access and share only with approved groups. Require owner, security/data, test, support, and funding approval. GCC organization-wide Teams and Microsoft 365 publication uses admin submission. [Share agents safely >](#)
- 8

Monitor, retain, and respond

For standalone agents, set transcript recording, download, retention, and viewer access; enable Dataverse auditing. For Microsoft 365 agents, use GCC-supported Purview controls. Document containment and rollback actions. [Transcript controls >](#)
- 9

Fund, meter, and review

Choose prepaid, PAYG, or a hybrid based on demand and continuity. Assign a capacity owner and cost center by environment; configure overage routing, alerts, and per-agent limits where available. Review consumption monthly. [Manage credits and capacity >](#)

Minimum production gate

- ✓

Primary and backup owners
- ✓

Approved audience and risk
- ✓

Entra authentication
- ✓

DLP policy passes
- ✓

Versioned solution release
- ✓

Security and UAT evidence
- ✓

Support and incident path
- ✓

Funding and limit decision

Plan billing, capacity, and guardrails

Choose the funding path and overage behavior before production use.

1. Choose the funding model

Prepaid packs provide a monthly tenant pool. **PAYG** bills consumed credits to Azure. A **hybrid** uses prepaid capacity first and PAYG for overage. [Licensing options >](#)

2. Allocate and route usage

Assign prepaid credits from the tenant pool to environments. At exhaustion, choose whether an environment can draw from remaining tenant capacity, bill a linked PAYG plan, or have no fallback. [Capacity allocation >](#)

3. Decide how usage stops

An environment allocation isn't an exact cap. With no fallback, prepaid usage enters overage and enforcement can disable agents; the monthly count resets rather than reducing the next allocation. PAYG continues billing until stopped. [Overage behavior >](#)

Guardrails: Configure consumption notifications and monthly per-agent hard stops where available. Azure budgets provide cost alerts but don't stop PAYG usage. For agency chargeback, isolate workloads by environment and billing plan. Confirm PAYG and newer capacity controls in the target GCC tenant. [Azure budgets >](#)